UE : Prog WEB

Compte rendu du projet prog web

Ceci est un rapport commenté et expliqué en détail sur la manière dont est traité mon projet de A jusqu'à Z...

Ce projet a été réalisé par :

Nom : RAAB

Prénom : Rayane

N° étudiant : 22418402

Année Universitaire : 2025/2026

I. Partie 1 : Micro-service en local :

Objectif : Le but de cette première partie est de réaliser un premier micro-service REST respectant la consigne "un seul service en local". Technologies imposées : Java Spring Boot + Docker + Kubernetes/Minikube.

Service choisi : Répertoire de Contacts (CRUD : nom, prénom, téléphone, émail) - API REST pure sans interface web (frontend = étape future).

Architecture réalisée :

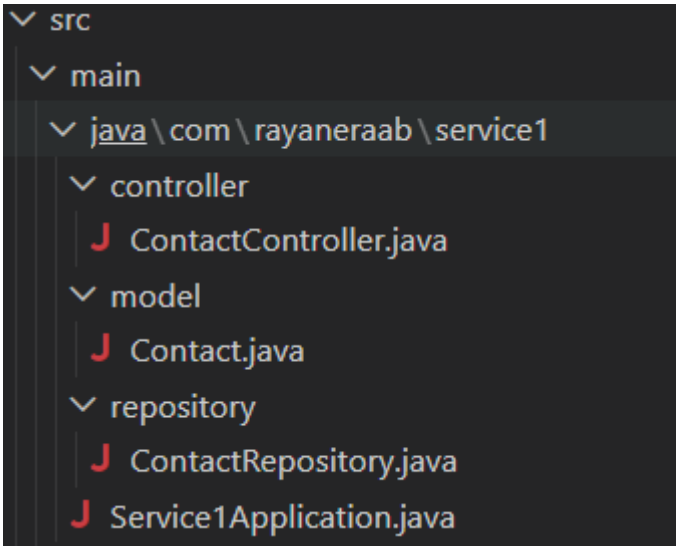
Service 1 (Spring Boot REST API)

-> Dockerisé et puis publié sur Docker Hub.
-> Déployé Kubernetes (Deployment + Service).
-> Accessible via Minikube (port-forward 8080).

Etapes techniques détaillés :

- Création micro-service Spring Boot : Le projet est généré à partir de start.spring.io sur VScode en adaptant les critères suivants :
  - ➔ Maven project, Java 21 (jdk-21), Spring Boot v4.0.3
  - ➔ Dépendences : Spring Web, Devtools, H2 (en mémoire)
  - ➔ Package : com.rayaneraab.service1

- Structure Projet standard :



→ Rest Endpoints

→ Record POJO

→ In-memory CRUD

→ App principale

- **Endpoints REST CRUD :**

GET	/api/contacts	→ Liste tous les contacts
POST	/api/contacts	→ Ajouter un contact
GET	/api/contacts/X	→ Récupérer le contact ayant pour id X
PUT	/api/contacts/X	→ Modifier le contact ayant pour id X
DELETE	/api/contacts/X	→ Supprimer le contact ayant pour id X

+ configuration fichier application.propreties → server.port = 8080

- **Conteneurisation Docker :** Commençons par créer le fichier Dockerfile tel que

Dockerfile X service1-0.0.1-SNAPSHOT.jar

Dockerfile > ...

```
1  FROM eclipse-temurin:21-jdk
2  VOLUME /tmp
3  EXPOSE 8080
4  ADD target/service1-0.0.1-SNAPSHOT.jar app.jar
5  ENTRYPOINT ["java","-Djava.security.egd=file:/dev/./urandom","-jar","/app.jar"]
```

Ensuite on procède au JAR build afin de nettoyer le dossier target, compiler le package en JAR. Ceci produit target/service1-0.0.1-SNAPSHOT.jar prêt pour Docker.

C:\Windows\System32\cmd.e

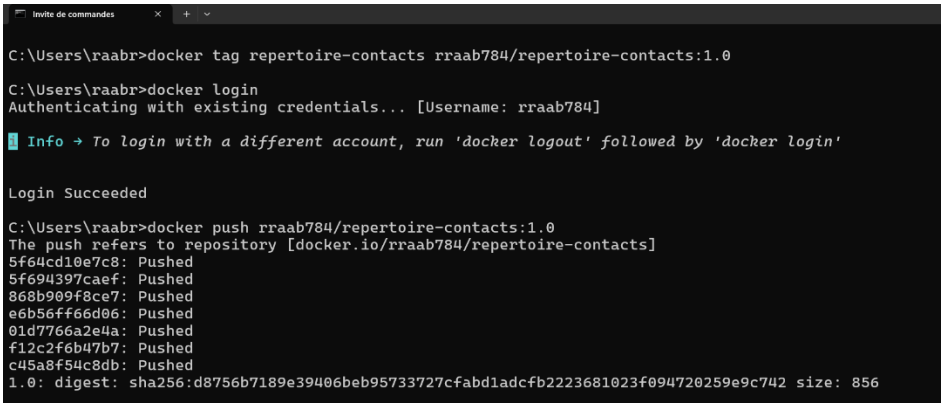
```
C:\Users\raabr\OneDrive\Bureau\service1>mvnw clean package -DskipTests
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< com.rayaneraab:service1 >-----------------------
[INFO] Building service1 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- clean:3.5.0:clean (default-clean) @ service1 ---
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ service1 ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.14.1:compile (default-compile) @ service1 ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 4 source files with javac [debug parameters release 21] to target\classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ service1 ---
[INFO] skip non existing resourceDirectory C:\Users\raabr\OneDrive\Bureau\service1\src\test\resources
[INFO]
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ service1 ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug parameters release 21] to target\test-classes
[INFO]
[INFO] --- surefire:3.5.4:test (default-test) @ service1 ---
[INFO] Tests are skipped.
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ service1 ---
[INFO] Building jar: C:\Users\raabr\OneDrive\Bureau\service1\target\service1-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:4.0.3:repackage (repackage) @ service1 ---
[INFO] Replacing main artifact C:\Users\raabr\OneDrive\Bureau\service1\target\service1-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\Users\raabr\OneDrive\Bureau\service1\target\service1-0.0.1-SNAPSHOT.jar.original
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.459 s
[INFO] Finished at: 2026-02-20T08:08:03+01:00
[INFO] ------------------------------------------------------------------------
```

Et puis, on fait le build qui construit une image Docker nommée repertoire-contacts à partir du Dockerfile du répertoire courant tel que :

C:\Windows\System32\cmd.e

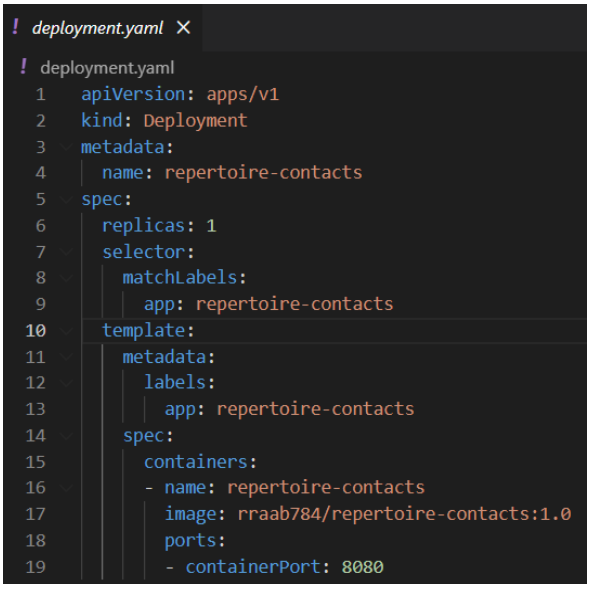
```
C:\Users\raabr\OneDrive\Bureau\service1>docker build -t repertoire-contacts .
[+] Building 1.7s (1/2)                                          docker:desktop-linux
 => [internal] load build definition from Dockerfile                              0.1s
[+] Building 1.9s (1/2)                                          docker:desktop-linux
 => [internal] load build definition from Dockerfile                              0.1s
[+] Building 2.0s (1/2)                                          docker:desktop-linux
 => [internal] load build definition from Dockerfile                              0.1s
[+] Building 2.2s (1/2)                                          docker:desktop-linux
 => [internal] load build definition from Dockerfile                              0.1s
[+] Building 3.0s (3/6)
 => [internal] load build context                                                 1.5s
 => => transferring context: 19.72MB                                              1.5s
[+] Building 6.[+] Building 37.4s (7/7) FINISH
ED                                                   docker:desktop-linux
 => [internal] load build definition from Dockerfile                              0.1s
 => => transferring dockerfile: 200B                                              0.0s
 => [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk         2.8s
 => [internal] load .dockerignore                                                 0.0s
 => => transferring context: 2B                                                   0.0s
 => [internal] load build context                                                 1.5s
 => => transferring context: 19.72MB                                              1.5s
 => [1/2] FROM docker.io/library/eclipse-temurin:21-jdk@sha256:c880d7bdbcab3bb1e82159d5781760832e94b72b65dfa1835b4c23  30.6s
 => => resolve docker.io/library/eclipse-temurin:21-jdk@sha256:c880d7bdbcab3bb1e82159d5781760832e94b72b65dfa1835b4c23b  0.1s
 => => sha256:c45a8f54c8db9e11d290f2107825fe437f540a499ffbd9b6f1cacdc720e6513c 2.28kB / 2.28kB  0.7s
 => => sha256:e6b56ff66d06b2f934e91725f7c5fb8077614763b978608b6ec116137cb910a1 158B / 158B  0.8s
 => => sha256:f12c2f6b47b7df253d03d4086c043b302e87c4d2b15e486ecf2ffb4cc12ada74 157.87MB / 157.87MB  25.8s
 => => sha256:868b909f8ce74fe7ddc783497c20e0a48df48b9f555fc57049945f9c47039c95 22.96MB / 22.96MB  6.7s
 => => sha256:01d7766a2e4a62b74e0bebf2cd12c47e675e9221174f6570854203e84ffe68b0 29.73MB / 29.73MB  26.4s
 => => extracting sha256:01d7766a2e4a62b74e0bebf2cd12c47e675e9221174f6570854203e84ffe68b0  0.8s
 => => extracting sha256:868b909f8ce74fe7ddc783497c20e0a48df48b9f555fc57049945f9c47039c95  0.7s
 => => extracting sha256:f12c2f6b47b7df253d03d4086c043b302e87c4d2b15e486ecf2ffb4cc12ada74  1.7s
 => => extracting sha256:e6b56ff66d06b2f934e91725f7c5fb8077614763b978608b6ec116137cb910a1  0.0s
 => => extracting sha256:c45a8f54c8db9e11d290f2107825fe437f540a499ffbd9b6f1cacdc720e6513c  0.0s
 => [2/2] COPY ./target/* app.jar                                                 2.5s
 => exporting to image                                                            1.2s
 => => exporting layers                                                           0.9s
 => => exporting manifest sha256:01b00d8c791ab0c249ec951ed918e477ef54370a20de48a195374aa6f3a44235  0.0s
 => => exporting config sha256:ad9d155e4a4b9ca0abef75c1242872c1141cd1c52a0fcde582ad1942f6a9d8a7  0.0s
 => => exporting attestation manifest sha256:4daeda7c79cfc4c55eeb5f34ec3402fcc483a108e477411201b1e31e5a2f20f5  0.0s
 => => exporting manifest list sha256:cee5dd3842f5484da6fa2bf11b29ec67af3d16899c2db88892c4de65a23d7c14  0.0s
 => => naming to docker.io/library/repertoire-contacts:latest                     0.0s
 => => unpacking to docker.io/library/repertoire-contacts:latest                  0.2s
```

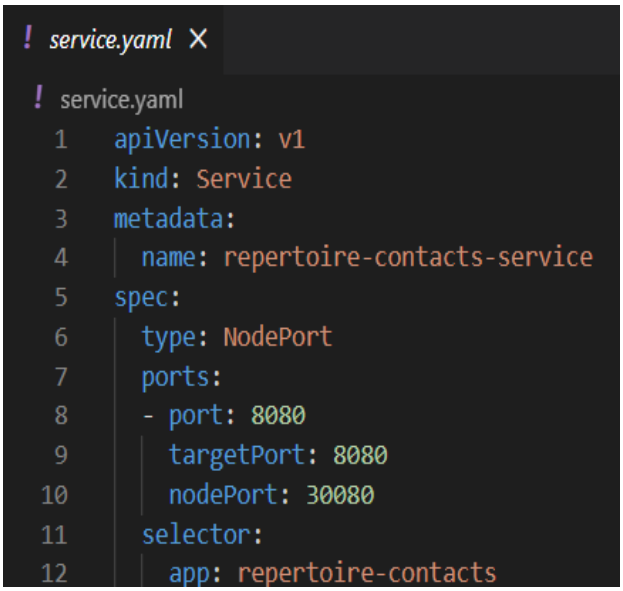
- **Publication Docker Hub :**



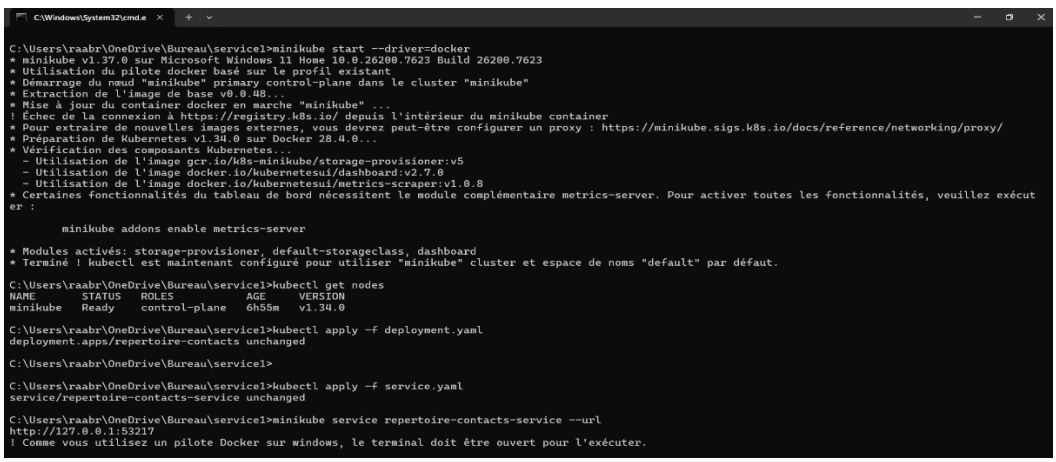
L'image publique est désormais **rraab784/repertoire-contacts:1.0**

- **Déploiement Kubernetes (Minikube) :** On commence par créer les deux fichiers YAML dont deployment.yaml et service.yaml

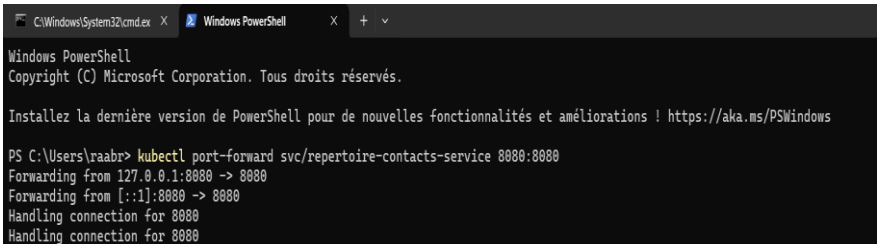




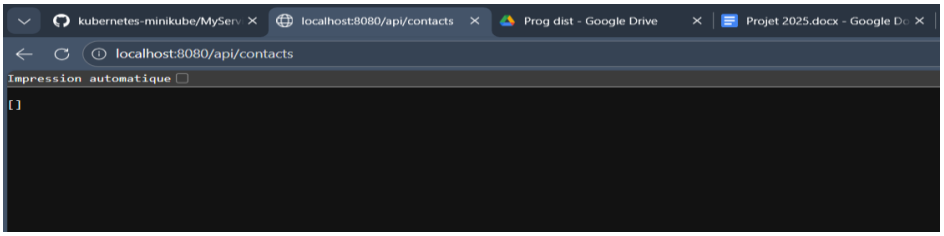
Ensuite, on réalise le déploiement kubernetes en mettant à jour une ressource K8s (Deployment / Service) depuis YAML de manière déclarative. Et en se procurant l'url externe pour accéder au service via Minikube (ouvre port NodePort).



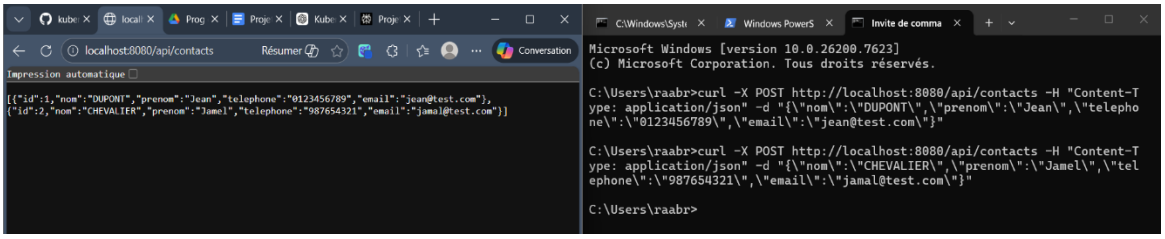
Ensuite, pour faciliter l'accès au service, on applique une redirection de port depuis le port 8080 lancé par minikube vers notre propre port 80 du localhost, ceci nous permettra de faciliter l'accès, désormais pour y accéder, on aura juste à taper l'url suivante : http://localhost:8080/api/contacts !



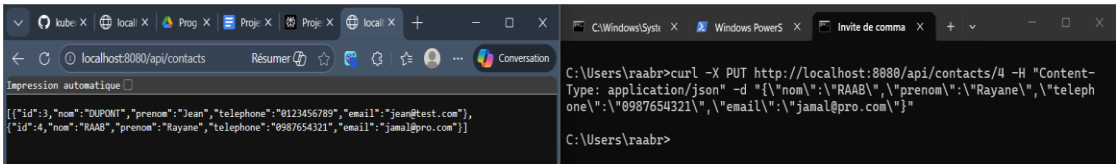
- **Tests fonctionnels réalisés :**



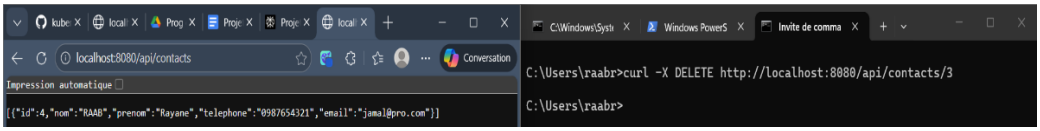
- En premier, on commence par tester le rajout des contacts dans un repertoire :



- Ensuite, on test la modification (update) d'un contact du repertoire :



- Enfin, on test la suppression d'un contact du repertoire :

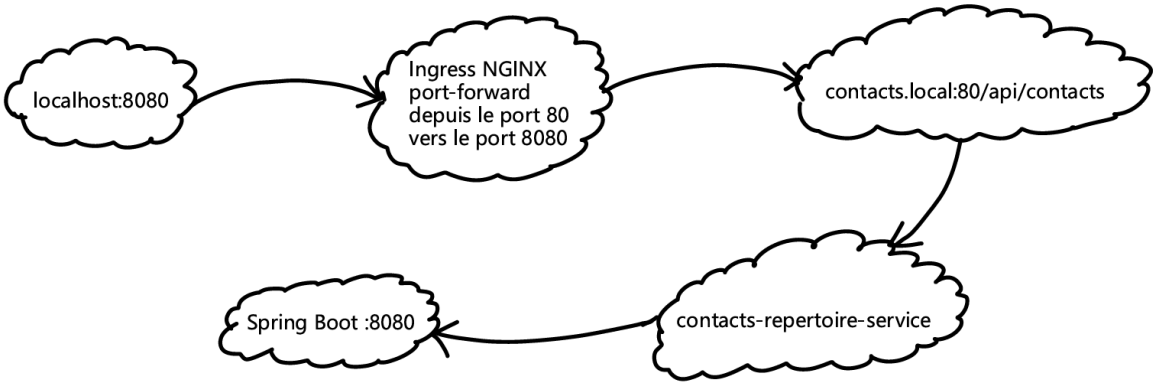


Objectif pleinement atteint. Microservice complet (code/Docker/K8s) avec tests CRUD validés. Architecture production-ready (image Hub, manifests YAML réutilisables). Base solide pour microservices multi-services.

II. Partie 2 : API Gateway :

Objectif : Le but de cette partie consiste à mettre en place une Gateway via Ingress NGINX devant le premier service Contacts développé lors de l'étape 1, ceci consiste à réaliser un routage *contacts.local/api/** vers notre service Contacts sur port 8080.

Architecture implémentée :



Étapes Techniques Détaillées :

- **Activation Ingress Controller** :

```
C:\Users\raabr\OneDrive\Bureau\service1>minikube addons enable ingress
* ingress est un addon maintenu par Kubernetes. Pour toute question, contactez minikube sur GitHub.
Vous pouvez consulter la liste des mainteneurs de minikube sur : https://github.com/kubernetes/minikube/blob/master/OWNERS
* Après que le module est activé, veuiller exécuter "minikube tunnel" et vos ressources ingress seront disponibles à "127.0.0.1"
  - Utilisation de l'image registry.k8s.io/ingress-nginx/controller:v1.13.2
  - Utilisation de l'image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
  - Utilisation de l'image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
* Vérification du module ingress...
* Le module 'ingress' est activé

C:\Users\raabr\OneDrive\Bureau\service1>kubectl get pods -n ingress-nginx
NAME                                        READY   STATUS      RESTARTS   AGE
ingress-nginx-admission-create-tzjqt        0/1     Completed   0          162m
ingress-nginx-admission-patch-nt5j6         0/1     Completed   1          162m
ingress-nginx-controller-9cc49f96f-hk7ds    1/1     Running     0          162m
```

→ Première commande servant à activer le controller ingress.

→ Deuxième commande pour s'assurer que le controller est bien actif (On peut voir ingress-nginx-controller… ayant 1/1 en READY et Running en STATUS).

- **Ingress routing by ingress.yaml :**

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: contacts-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: contacts.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: repertoire-contacts-service
            port:
              number: 80
```

On commence par créer le fichier **ingress.yaml**, ce fichier contenant la config de notre ingress, notamment le Host : **contacts.local**, et le service **repertoire-contacts-service** qui sera exposé sur le port **80.**

```
C:\Users\raabr\OneDrive\Bureau\service1>kubectl get ingress
No resources found in default namespace.

C:\Users\raabr\OneDrive\Bureau\service1>kubectl apply -f ingress.yaml
ingress.networking.k8s.io/contacts-ingress created

C:\Users\raabr\OneDrive\Bureau\service1>kubectl get ingress
NAME               CLASS   HOSTS             ADDRESS   PORTS   AGE
contacts-ingress   nginx   contacts.local              80      2s

C:\Users\raabr\OneDrive\Bureau\service1>kubectl get node -o wide
NAME       STATUS   ROLES           AGE    VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION                     CONTAINER-RUNTIME
minikube   Ready    control-plane   174m   v1.34.0   192.168.58.2   <none>        Ubuntu 22.04.5 LTS   6.6.87.2-microsoft-standard-WSL2   docker://28.4.0
```

Procédure pour mettre en marche notre ingress.yaml

- **Accès au service via Port-Forward :** Ceci permet de créer un tunnel réseau entre notre local host et le service Ingress NGINX situé au niveau du cluster Minikube, cela nous permettra de récupérer tout le trafic exposé par le service Ingress sur le port 80 et de le rediriger vers le port 8080 de notre localhost.

```
C:\Users\raabr\OneDrive\Bureau\service1>kubectl port-forward svc/ingress-nginx-controller -n ingress-nginx 8080:80 --address 0.0.0.0
Forwarding from 0.0.0.0:8080 -> 80
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
```

- **DNS Local :** Pour permettre la résolution du nom de domaine contacts.local via le fichier hosts Windows vers son @ip correspondante qui est bel et bien celle de notre localhost (127.0.0.1) où sera redirigé tout le trafic du port 80 venant du service Ingress.

  Procédure à suivre :

  → Bloc-notes à exécuter en tant qu'administrateur.

  → Cliquer sur Fichier, Ouvrir et taper C:\Windows\System32\drivers\etc\hosts.

  → Ajouter la ligne **127.0.0.1 contacts.local**, et puis enregistrer.

- **Tests Fonctionnels Réalisés :**

```
C:\Users\raabr\OneDrive\Bureau\service1>curl -X GET  http://contacts.local:8080/api/contacts
[{"id":1,"nom":"raab","prenom":"rayane","telephone":"039329394","email":"rak@gmail.com"},{"id":2,"nom":"FABIEN","pren
om":"Nicolas","telephone":"0123456789","email":"nico.fab@gmail.com"},{"id":3,"nom":"TEST","prenom":"Nginx","telephone
":"0123456789","email":"test.nginx@gmail.com"}]
C:\Users\raabr\OneDrive\Bureau\service1>
C:\Users\raabr\OneDrive\Bureau\service1>curl -X GET  http://contacts.local:8080/api/contacts/3
{"id":3,"nom":"TEST","prenom":"Nginx","telephone":"0123456789","email":"test.nginx@gmail.com"}
C:\Users\raabr\OneDrive\Bureau\service1>
C:\Users\raabr\OneDrive\Bureau\service1>curl -X POST -H "Content-Type: application/json" http://contacts.local:8080/a
pi/contacts -d "{\"nom\":\"TEST1\", \"prenom\":\"Nginx2\", \"telephone\":\"0123456789\", \"email\":\"test1.nginx2@gma
il.com\"}"

C:\Users\raabr\OneDrive\Bureau\service1>curl -X GET  http://contacts.local:8080/api/contacts
[{"id":1,"nom":"raab","prenom":"rayane","telephone":"039329394","email":"rak@gmail.com"},{"id":2,"nom":"FABIEN","pren
om":"Nicolas","telephone":"0123456789","email":"nico.fab@gmail.com"},{"id":3,"nom":"TEST","prenom":"Nginx","telephone
":"0123456789","email":"test.nginx@gmail.com"},{"id":4,"nom":"TEST1","prenom":"Nginx2","telephone":"0123456789","emai
l":"test1.nginx2@gmail.com"}]
```

- **Problèmes Rencontrés & Solutions :**

  Port 80 Windows bloqué :

- *ERREUR:* "bind: Only one usage of each socket address (protocol/network address/port)"
- *CAUSE:* Windows réserve port 80 (IIS/HTTP.sys, HTTP.sys)
- *SOLUTION:* Port-forward 8080:80 → localhost:8080

  Minikube tunnel problem because of Windows port 80 blocked :

ERREUR : minikube tunnel → Port 80 Windows toujours bloqué → Remplacé par port-forward Ingress controller

III. Partie 3 : Deuxième micro-service en local :

Contexte et objectif : Dans le cadre de notre architecture micro-services, nous avons développé un service web frontend qui constitue la deuxième composante de notre application de gestion de répertoire de contacts. Ce service frontend s'appuie entièrement sur notre premier service (API Backend) déjà déployé comme service 1, qui expose une API REST complète pour les opérations CRUD sur les contacts. Et donc, le rôle des deux services devient désormais :

- **Service 1 :** qui est le service backend, contient une API REST /api/contacts pour la persistance et logique métier.
- **Service 2 :** qui constitue le service frontend, contient une Interface web utilisateur pour consommer notre API backend d'une façon simple et fluide.

Construction du Service Frontend : Le frontend a été développé avec Spring Boot + Thymeleaf pour offrir une interface web moderne et réactive. Et en ce qui concerne les étapes de développement de ce service, on aura :

- **Génération du projet :** avec spring initializr avec dépendances Web Spring, Thymeleaf et WebFlux (Rest template).
- **Contrôleur principal :** FrontendController avec 4 endpoints CRUD dont :
  - **Endpoint READ (GET /) pour lister tous les contacts**

```java
// Liste tous les contacts - Page principale
@RequestMapping(value = "/", method = RequestMethod.GET)
public String home(Model model) {
    try {
        Contact[] contacts = restTemplate.getForObject(backendUrl, responseType: Contact[].class);
        model.addAttribute(attributeName: "contacts", contacts != null ? Arrays.asList(contacts) : List.of());
        model.addAttribute(attributeName: "message", attributeValue: "✅ Backend connecté");
    } catch (Exception e) {
        model.addAttribute(attributeName: "contacts", List.of());
        model.addAttribute(attributeName: "error", "Backend indisponible: " + e.getMessage());
    }
    return "index";
}
```

  - **Endpoint CREATE (POST /add) pour ajouter un contact**

```java
// Ajouter contact
@RequestMapping(value = "/add", method = RequestMethod.POST)
public String addContact(
        @RequestParam String nom,
        @RequestParam String prenom,
        @RequestParam String telephone,
        @RequestParam String email,
        RedirectAttributes redirectAttributes) {

    try {
        Contact newContact = new Contact(id: null, nom, prenom, telephone, email);
        restTemplate.postForObject(backendUrl, newContact, responseType: Void.class);
        redirectAttributes.addFlashAttribute(attributeName: "message", attributeValue: "Contact ajouté avec succès !");
    } catch (Exception e) {
        redirectAttributes.addFlashAttribute(attributeName: "error", "Erreur ajout: " + e.getMessage());
        return "redirect:/";
    }
    return "redirect:http://frontend.local:8080/";
}
```

  - **Endpoint UPDATE (POST /update/{id}) pour modifier un contact**
  - **Endpoint DELETE (GET /delete/{id}) pour supprimer un contact**
- **Templates HTML :** dont index.html (une page HTML qui liste tous les contacts et propose une multitude de choix entre ajout, suppression et modification des

différents contacts), et une deuxième page edit.html contenant un formulaire, cette page est utilisée lors de la modification d'un contact…

- **Configuration:** backend.api.url pointant vers service K8s Backend.

Dockerisation : Construction du Dockerfile tel que :

```
Dockerfile > ...
1   FROM eclipse-temurin:21-jdk
2   VOLUME /tmp
3   EXPOSE 8081
4   ADD target/simple-springboot-webapp-1.0-SNAPSHOT.jar app.jar
5   ENTRYPOINT ["java", "-Djava.security.egd=file:/dev/./urandom", "-jar", "app.jar"]
```

Et puis exécuter quelques commandes tel que suit :

- mvn clean package -DskipTests → pour compilation et packaging de l'application Spring Boot en JAR exécutable (clean pour supprimer les builds précédents notamment en supprimant le dossier target).
- docker build -t repertoire-frontend → pour construction de l'image Docker multistage.
- docker run repertoire-frontend → pour test local de l'image avant déploiement Kubernetes.

Publication sur Docker Hub : En tapant l'ensemble des commandes suivantes :

- docker login → pour se logger en tapant login + mot de passe
- docker tag repertoire-frontend rraab784/repertoire-contacts-frontend :1.0
- docker push rraab784/repertoire-contacts-frontend :1.0 → désormais, une image est rendue publique sur Docker Hub sous le lien suivant : https://hub.docker.com/r/rraab784/repertoire-contacts-frontend

Déploiement Kubernetes Minikube : Fichier deployment.yaml créé, ensuite le déploiement qui se fait de la manière suivante :

- minikube start –driver=docker → pour lancer Minikube.
- kubectl apply -f deployment.yaml service.yaml → pour deployer notre service frontend sous le nom repertoire-frontend-service.
- minikube service repertoire-frontend-service → pour lancer le service frontend sous minikube et faire les tests (avant configuration de la gateway).
- Ensuite on peut utiliser la commande sur l'image ci-dessous pour tester la connectivité entre frontend service ←et→ backend service.

Configuration Gateway Ingress NGINX : L'objectif est d'obtenir un accès unifié via domaines frontend.local (pour le service 2 frontend) et contacts.local (pour le service 1 backend). Les étapes à suivre consistent en :

- Minikube addons enable ingress → pour activer NGINX Controller

- Rédaction du fichier ingress.yaml pour service2 pour les règles de routage… (host : frontend.local → repertoire-frontend-service:80, host : contacts.local → repertoire-contacts-service:80)
- kubectl apply -f ingress.yaml → pour deployer l'ingress du frontend
- Fichier hosts Windows à modifier en rajoutant la ligne suivante en tête du fichier 127.0.0.1 frontend.local
- Redirection du port ingress, le port 80 n'est pas directement accessible depuis le cluster de notre service frontend à cause des politiques Windows, c'est pourquoi on se doit de faire une redirection du trafic sur ce port vers le port 8080 de notre machine localhost, d'où vient le 127.0.0.1 frontend.local ! pour effectuer ce forward on utilise la commande suivante :
    - Kubectl port-forward -n ingress-nginx svc/ingress-nginx-controller 8080:80 –address 0.0.0.0

- Grâce à cette commande, on crée un tunnel réseau entre le localhost:8080 et le service ingress-nginx-controller, cela redirige donc les deux types d'ingress déployés précédemment sur le port 8080 de notre localhost, autrement dit, tout passe par localhost:8080 et ingress route selon Host Header tel que :
    - http://frontend.local:8080 → pour la page liste contacts
    - http://contacts.local:8080/api/contacts → pour la JSON API

Tests effectués :

- **Test 1 : Accès gateway :**

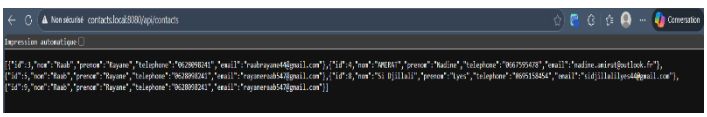
```
C:\Users\raabr\OneDrive\Bureau\service2>kubectl port-forward svc/ingress-nginx-controlle
r -n ingress-nginx 8080:80 --address 0.0.0.0
Forwarding from 0.0.0.0:8080 -> 80
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
```

Port forward 8080 : 80

Page liste contacts

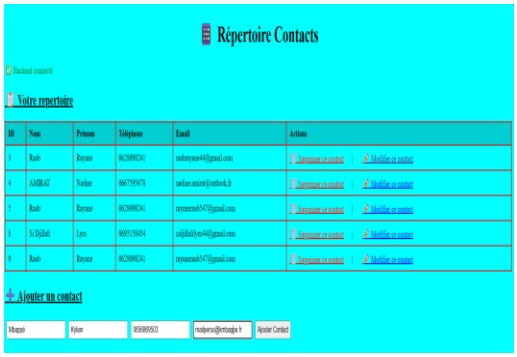
Service2 ✅



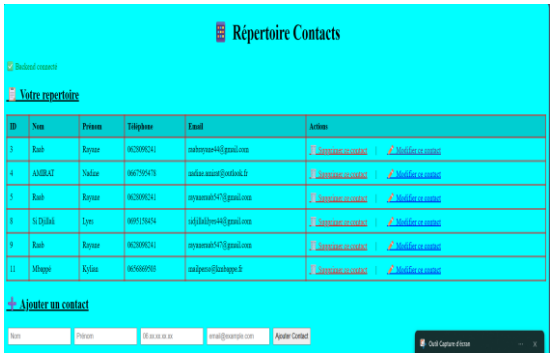


http://contacts.local:8080/api/contacts
JSON API → ✅

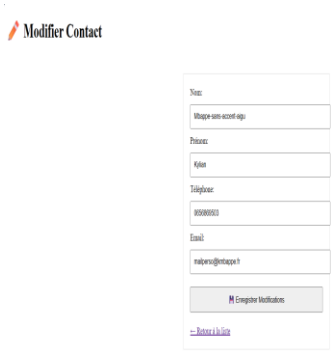
- **Test 2 : CRUD via Frontend :**



- Avant rajout -



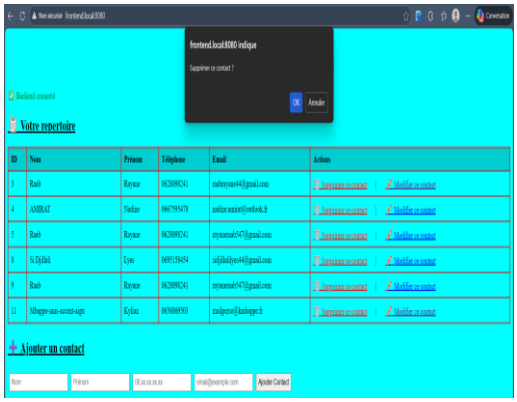
- Après rajout -



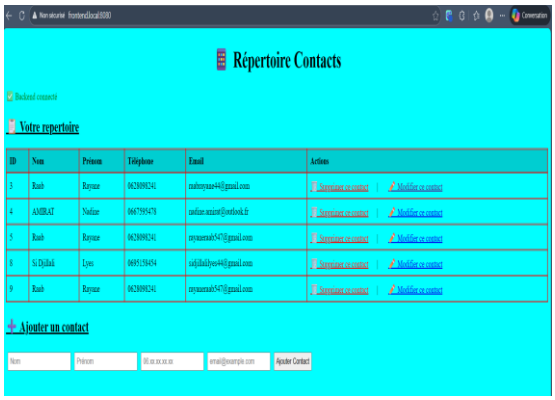
- Avant update sur nom -



- Après update sur nom -



- Avant suppression -



- Après suppression -

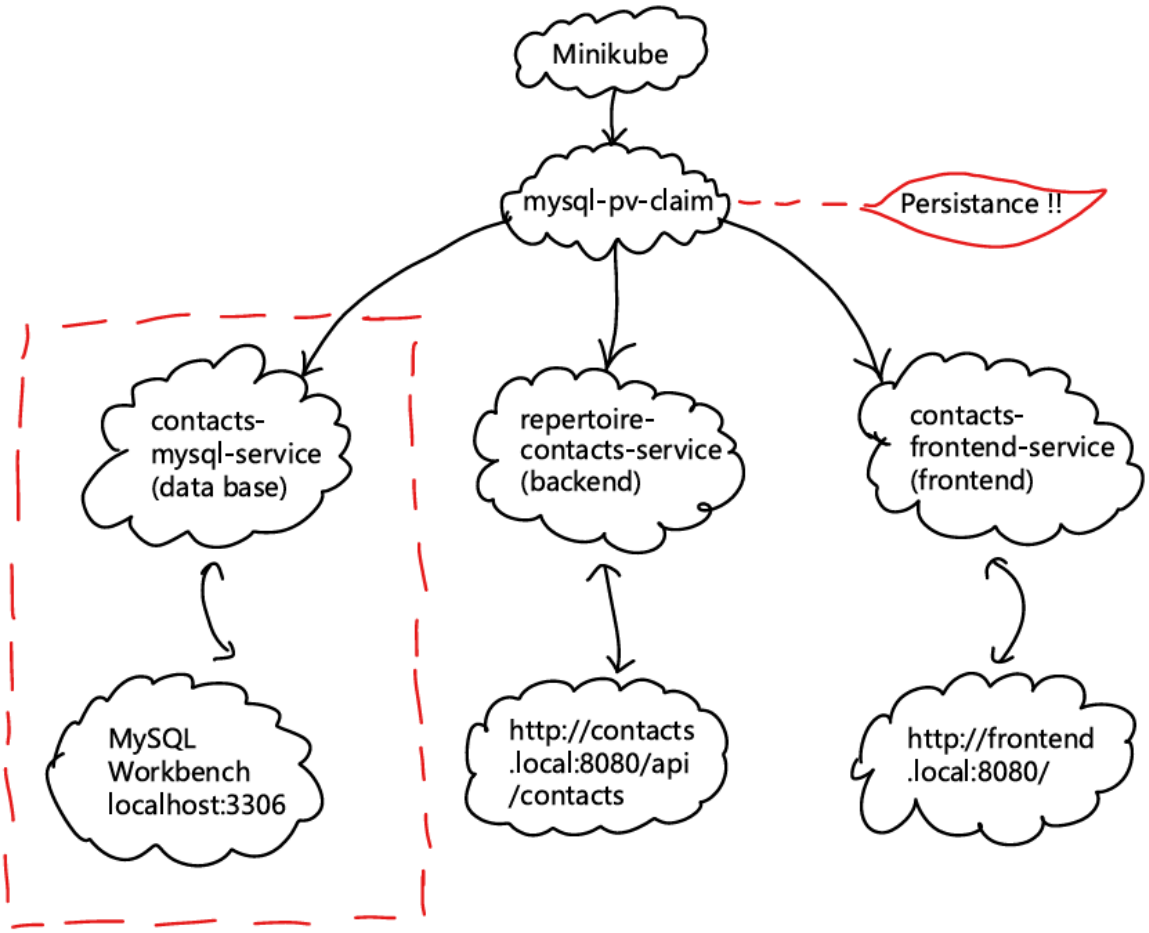
IV. Partie 4 : Ajout d'un service base de données sur Minikube (en local) :

Contexte et objectif : La mise en place d'un service MySQL persistant pour notre projet Répertoire-Contacts permettra :

- Stockage durable des contacts (nom, prénom, téléphone et émail).
- Connexion sécurisée depuis le backend Spring Boot.
- Visualisation du contenu de la data base via Workbench.
- Déploiement Kubernetes sur Minikube (environnement de développement).

Le produit final sera sous forme d'une base de données accessible, persistante et intégrée au backend…

Architecture finale :



Etapes techniques détaillées : Dans cette partie on va détailler les étapes d'intégration MySQL et de migration depuis H2 in-memory vers un SGBD plus persistant, plus sécurisé et surtout plus efficace…

Migration H2 in-memory vers MySQL : pour ce faire, on a dû changer le fichier de configuration application.properties situé au niveau du dossier ressources, en remplaçant la configuration précédente H2 in-memory par la configuration MySQL suivante :

```
# DataSource MySQL
spring.datasource.url=jdbc:mysql://mysql-service:3306/contacts_db?\
  useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC&useUnicode=true&characterEncoding=UTF-8
spring.datasource.username=contacts
spring.datasource.password=contacts123
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect

# HikariCP Pool
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.connection-timeout=30000

# Logging (debug)
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
```

Cette configuration a été appliquée après que les dépendances nécessaires on été rajoutées au niveau du fichier pom.xml afin d'éviter tout problème venant de l'utilisation d'un module externe non-importé… ces dépendances ont été ajoutées de la manière suivante :

```
<dependencies>

    <!-- JPA + Hibernate -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- MySQL Driver -->
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
```

Entité JPA Contact : L'entité JPA Contact est le cœur de la persistance des données, cette entité sert à mapper l'objet Java vers la table MySQL contacts (Hibernate traduit automatiquement Contact.java vers une table avec colonnes id, nom, prenom, telephone, email sur MySQL), cette entité permet également de gérer les opération CRUD (ajout, edit, delete, getAll) d'une manière automatique, on configure tout ça à partir du fichier Contact.java qui se trouve au niveau du dossier model en appliquant la config ci-dessous :

```java
@Entity
@Table(name = "contacts")
public class Contact {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nom;

    @Column(nullable = false, length = 100)
    private String prenom;

    @Column(name = "telephone", nullable = false, length = 20)
    private String telephone;

    @Column(nullable = false, length = 255, unique = true)
    private String email;

    // CONSTRUCTEUR VIDE JPA (OBLIGATOIRE)
    public Contact() {}

    // CONSTRUCTEUR COMPLET
    public Contact(Long id, String nom, String prenom, String telephone, String email) {
        this.id = id;
        this.nom = nom;
        this.prenom = prenom;
        this.telephone = telephone;
        this.email = email;
    }

    // GETTERS / SETTERS MANUELS (JPA)
```

On a également le fichier ContactRepository.java qui est en contact direct avec la base de données MySQL, ce fichier permet de gérer automatiquement les opérations CRUD tel que l'opération save(), delete(), findAll() ainsi que l'opération findById().

```java
package com.rayaneraab.service1.repository;

import com.rayaneraab.service1.model.Contact;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ContactRepository extends JpaRepository<Contact, Long> {
    // Hérite automatiquement : findAll(), save(), findById(), delete()
}
```

Manifestes Minikube : On configure tous nos fichiers de déploiement en un seul fichier mysql-statefulset.yaml. On commence par y insérer la partie PVC persistance.

mysql-statefulset.yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi  # Persistance données !
```

Cette partie décrit une demande de stockage persistant pour MySQL dans **Minikube** (Il crée un objet **PersistentVolumeClaim** qui demande un stockage de 5GO, ce volume est monté en lecture/écriture par un seul nœud à la fois, ce qui correspond parfaitement à un serveur MySQL

Ensuite vient la partie StatefulSet tel que :

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        env:
        - name: MYSQL_ROOT_PASSWORD
          value: root
        - name: MYSQL_DATABASE
          value: contacts_db
        - name: MYSQL_USER
          value: contacts
        - name: MYSQL_PASSWORD
          value: contacts123
        ports:
        - containerPort: 3306
        volumeMounts:
        - name: mysql-storage
          mountPath: /var/lib/mysql
  volumeClaimTemplates:
```

Ce fichier décrit un StatefulSet Kubernetes qui sert à déployer une base de données MySQL 8.0 de façon stable et persistante. Il indique qu'il faut 1 réplique du pod MySQL, avec les variables d'environnement nécessaires pour créer la base contacts_db et l'utilisateur contacts avec son mot de passe. Le bloc volumeMounts et volumeClaimTemplates permet de monter un volume persistant de1 Gi sur /var/lib/mysql, ce qui garantit que les données de la base sont conservées même si le pod est recréé ou redémarré.

Déploiement : Toutes les étapes de déploiement sont mentionnées ci-dessous :

minikube start –driver=docker

kubectl apply -f deployment.yaml

kubectl apply -f mysql-statefulset.yaml

Tests Base de Données : Pour effectuer les tests sur notre base de données, on utilise une méthode simple, notre méthode consiste à exposer le service de notre base de données sur le port 3306 de notre localhost, ensuite y accéder à notre database par le client MySQL (à installer si ce n'est pas encore fait) via un terminal cmd…

Avec cette première commande **kubectl get services**, on repère le nom de notre service base de données, qui est **mysql-service** dans mon cas.

La deuxième commande s'occupe de faire le port forwarding, ce terminal restera actif pour maintenir le port-forward (à ne surtout pas fermer ce terminal) ;

```
C:\Users\raabr>kubectl get services
NAME                         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
kubernetes                   ClusterIP   10.96.0.1       <none>        443/TCP          7d14h
mysql-service                ClusterIP   10.97.67.82     <none>        3306/TCP         4d11h
repertoire-contacts-service  ClusterIP   10.96.149.231   <none>        80/TCP           7d13h
repertoire-frontend-service  NodePort    10.98.84.168    <none>        80:30081/TCP     5d2h

C:\Users\raabr>kubectl port-forward svc/mysql-service -n default 3306:3306
Forwarding from 127.0.0.1:3306 -> 3306
Forwarding from [::1]:3306 -> 3306
Handling connection for 3306
```

Maintenant, plus qu'à me connecter à notre base de données via le client MySQL en utilisant un autre cmd à coté :



Désormais, on peut le dire, notre service base de données est fonctionnel !!

V. Liens utiles :

Image Docker Hub service 1 (Backend + DataBase):

rraab784/repertoire-contacts - Docker Image

Image Docker Hub service 2 (Frontend):

rraab784/repertoire-contacts-frontend - Docker Image

Code du projet (Java sur Github) :

RynRAAB/K8S-project: Projet M1 Cybersécurité - Kubernetes

VI. Récapitulatif de mes progrès sur Google skills pour nos Google-labs:

Labs réalisés :

- Get Started with Security Command Center
- Securing Container Builds
- A tour of Google Cloud Hands-On Labs
- Infrastructure as Code with Terraform
- How to Use a Network Policy on Google Kubernetes Engine

